Comet 浏览器与 OpenClaw 集成方案

基于意图注入与状态监控的 Agent-to-Agent 协作架构

执行摘要

本文档提出一种将 Perplexity Comet 浏览器集成到 OpenClaw AI Agent 框架的技术方案。核心理念是：**不重复实现 Comet 已有的 Agentic Browsing 能力，而是通过 Chrome DevTools Protocol (CDP) 实现"意图注入"和"状态监控"**，让 OpenClaw 能够调用 Comet 的原生智能浏览功能进行 Web 搜索和深度研究任务。

相比传统的 DOM 节点点击自动化方案，该架构具有更强的 UI 改版抗性、更低的维护成本，以及对 Comet Agent 能力的最大化利用。

架构概览

系统组成

整体架构采用三层设计：

Figure 1: Comet-OpenClaw 集成架构图

1. **OpenClaw Agent Loop** — 负责任务规划和 Skill 调度
2. **Perplexity Skill (CDP 控制层)** — 通过 CDP 与 Comet 通信
3. **Comet Browser** — 执行实际的浏览和 AI 推理任务

数据流向

OpenClaw 下发任务指令
↓
[意图注入层] 将自然语言任务注入 Comet Assistant 输入框
↓
Comet AI 自主规划执行路径（云端 AI + 本地 Agent）
↓
[状态监控层] 实时拦截 SSE/WebSocket 通道获取执行状态
↓
[结果提取层] 任务完成后从 DOM 提取结构化结果
↓
返回给 OpenClaw 用于后续任务

技术实现细节

1. CDP 连接建立

Comet 基于 Chromium 内核，支持标准 CDP 协议[1]。启动时需开启远程调试端口：

macOS 启动命令：
```
/Applications/Comet.app/Contents/MacOS/Comet
--remote-debugging-address=127.0.0.1
--remote-debugging-port=9222
--no-first-run
```

Windows 启动命令：
```
"%LOCALAPPDATA%\Perplexity\Comet\Application\comet.exe"
--remote-debugging-address=127.0.0.1
--remote-debugging-port=9222
```

启动后可通过 `http://localhost:9222/json` 验证 CDP 端点是否正常工作。

Playwright 连接代码：
```
from playwright.async_api import async_playwright

async def connect_to_comet():
p = await async_playwright().start()
```

browser = await p.chromium.connect_over_cdp("http://localhost:9222")
context = browser.contexts[0] # 复用已登录的用户 session
return browser, context

关键优势：直接复用 Comet 中已有的 Perplexity 登录状态，无需处理鉴权[2]。

2. 意图注入层

设计原则： 不模拟点击 UI 元素，而是直接操作 React 内部状态，将任务指令注入到 Comet Assistant 的输入通道。

```
async def inject_intent(page, task: str, mode: str = "search"):
"""
注入任务意图到 Comet Assistant

Args:
    task: 自然语言任务描述
    mode: "search" | "deep_research" | "agent_task"
"""
# 通过 React 内部状态直接设置输入值
await page.evaluate("""
    (task) => {
        const input = document.querySelector('[data-testid="assistant-input"]');
        const nativeInputSetter = Object.getOwnPropertyDescriptor(
            window.HTMLTextAreaElement.prototype, 'value'
        ).set;
        nativeInputSetter.call(input, task);
        input.dispatchEvent(new Event('input', { bubbles: true }));
    }
""", task)

# Deep Research 模式切换
if mode == "deep_research":
    await page.evaluate("""
        document.querySelector('[data-mode="deep-research"]')?.click()
    """)
```

```
# 提交任务
await page.keyboard.press("Enter")
```

技术亮点： 通过劫持 `HTMLTextAreaElement.prototype` 绕过 React 的 SyntheticEvent 系统，确保状态更新正确触发[3]。

3. 状态监控层（核心创新）

传统自动化方案通过轮询 DOM 判断任务完成，存在以下问题：

- 无法获取中间状态
- 误判完成信号（骨架屏、加载动画）
- 高 CPU 开销

本方案采用网络层监控，直接拦截 Comet 与 Perplexity 服务端的通信通道。

3.1 SSE 流拦截（获取推理结果）

Comet 通过 Server-Sent Events (SSE) 接收 AI 生成的回答内容[4]：

```
async def monitor_sse_stream(page) -> AsyncGenerator[str, None]:
"""拦截 Perplexity SSE 数据流，实时获取推理过程"""
```

```
result_chunks = []
task_complete = asyncio.Event()

async def handle_sse_route(route):
    response = await route.fetch()
    body = await response.text()

    # 解析 SSE 格式数据
    for line in body.split("\n"):
        if line.startswith("data:"):
            chunk = json.loads(line[5:])

            if chunk.get("type") == "answer_chunk":
                result_chunks.append(chunk["text"])
```

```
            yield chunk["text"]

        elif chunk.get("type") == "task_complete":
            task_complete.set()

    await route.fulfill(response=response)

await page.route("**/rest/sse/**", handle_sse_route)
await page.route("**/api/answer**", handle_sse_route)

await task_complete.wait()
return "".join(result_chunks)
```

3.2 WebSocket 监控（Agent 执行状态）

Comet 的 Agent 执行通过 WebSocket (`wss://pplx.ai/agent`) 进行高频 RPC 通信[4]:

```
async def monitor_agent_websocket(page):
"""监控 comet-agent 的 WebSocket 通信"""

def on_websocket(ws):
    def on_frame_received(payload):
        msg = json.loads(payload)

        # RPC 执行状态
        if msg.get("method") == "action_result":
            print(f"✅ Agent 执行: {msg['params']['action']} "
                  f"→ {msg['params']['status']}")

        # Deep Research 进度
        if msg.get("type") == "research_progress":
            print(f"□ 研究进度: {msg['step']}/{msg['total_steps']}")

    ws.on("framereceived", lambda p: on_frame_received(p))

page.on("websocket", on_websocket)
```

关键优势： 直接从网络层获取 AI 决策和执行状态，无需解析 UI 变化。

4. 结果提取层

任务完成信号触发后，仅需一次 DOM 查询即可获取结构化结果：

```
async def extract_result(page) -> dict:
"""提取任务执行结果"""
return await page.evaluate("""
() => ({
answer: document.querySelector('.prose')?.innerText,
citations: [...document.querySelectorAll('a[data-citation]')]
.map(a => ({ text: a.innerText, url: a.href })),
research_steps: [...document.querySelectorAll('.research-step')]
.map(s => s.innerText)
})
""")
```

OpenClaw Skill 封装

将上述能力封装为标准 OpenClaw Skill：

```
class CometPerplexitySkill:
"""
OpenClaw Skill: 将 Comet 的 Agentic Browsing 能力暴露为标准接口
"""
```

```
MANIFEST = {
    "name": "comet_perplexity",
    "description": "Delegates search/research tasks to Comet browser's native A
    "tools": [
        {
            "name": "search",
            "description": "Fast web search with citations (10-30 seconds)",
            "parameters": {"query": "string"}
        },
        {
            "name": "deep_research",
```

```
            "description": "Multi-step research report (3-5 minutes)",
            "parameters": {"query": "string"}
        },
        {
            "name": "agent_task",
            "description": "General agentic task: book, compare, extract data, etc.",
            "parameters": {"query": "string"}
        }
    ]
}

async def run(self, tool: str, query: str) -> dict:
    """执行任务的统一入口"""
    async with self._get_comet_page() as page:

        # 1. 注入意图（自然语言，不操作具体 UI）
        await inject_intent(page, query, mode=tool)

        # 2. 并行监控两条通道
        sse_task = asyncio.create_task(monitor_sse_stream(page))
        ws_task = asyncio.create_task(monitor_agent_websocket(page))

        # 3. 等待任意完成信号
        done, _ = await asyncio.wait(
            [sse_task, ws_task],
            return_when=asyncio.FIRST_COMPLETED
        )

        # 4. 提取结构化结果
        return await extract_result(page)

async def _get_comet_page(self):
    """管理浏览器生命周期"""
    browser, context = await connect_to_comet()
    page = await context.new_page()
    await page.goto("https://www.perplexity.ai/")
    return page
```

使用示例：

OpenClaw 任务编排中

skill = CometPerplexitySkill()

快速搜索

result = await skill.run("search", "2026年AI发展趋势")

深度研究

report = await skill.run("deep_research", "量子计算在药物研发中的应用")

方案优势分析

对比传统 DOM 自动化方案

维度	DOM 点击方案	意图注入方案
UI 改版抗性	弱（selector 频繁失效）	强（只依赖输入框和网络层）
对 Agent 能力利用	完全绕过 Comet Agent	完全复用 Comet Agent
Deep Research 支持	需手动模拟每一步	Agent 自主执行
代码复杂度	高（几百行 selector 维护）	低（注入 + 监听）
登录态管理	需单独处理 cookie	复用 Comet 现有 session
实时状态感知	无（只能轮询 DOM）	有（网络层实时流）

Table 1: 方案对比表

核心技术优势

- **Agent-to-Agent 协作模式** — OpenClaw 只负责"what"（目标），Comet AI 负责"how"（执行路径），避免重复实现 AI 决策逻辑
- **网络层监控** — 绕过前端渲染的不确定性，直接从数据源获取状态
- **最小化 UI 依赖** — 仅依赖输入框这一个稳定元素，其他全在网络层
- **原生能力复用** — 充分利用 Comet 的多步推理、自动引用、截图理解等高级特性[5]

安全性与合规性分析

Comet 安全防护机制

Perplexity 在 Comet 中实现了多层安全防护，主要针对 Prompt Injection 攻击[6][7]：

1. **ML 分类器** — 实时检测网页内容中的恶意指令
2. **结构化 Prompting** — 在 system prompt 中标记外部内容为"不可信"
3. **敏感操作确认** — 发送邮件、修改账户等需用户明确确认
4. **透明通知** — 触发防护时向用户展示详细信息

本方案的威胁模型

关键区别： Comet 的安全防护针对"外部网页内容攻击 AI"，而本方案是"本地用户通过 CDP 控制自己的浏览器"。

操作来源	Comet 信任级别	触发安全防护
外部网页内容	不可信	√
CDP Runtime.evaluate	本地用户操作	✗
CDP 网络拦截	本地流量监听	✗
注入输入框的 query	用户指令	✗

Table 2: 安全机制触发条件

结论： 通过 CDP 执行的操作在 Comet 看来等同于用户坐在屏幕前手动操作，不会触发任何安全防护机制[8]。

实际风险与缓解措施

- **速率限制** — Perplexity 服务端可能对频繁自动化请求进行限流
  - 缓解：在 Skill 层实现请求队列和间隔控制
- **前端代码变化** — React 内部状态注入方式可能随版本更新失效
  - 缓解：优先使用 Playwright 标准 API（page.fill, page.press）
- **WebSocket 加密** — 部分 RPC 通信可能使用加密帧
  - 缓解：SSE 流已足够获取完整结果，WebSocket 仅用于可选的进度监控

实施路线图

Phase 1: 最小可行原型（1-2 天）

1. 手动启动 Comet 并开启 CDP 端口 9222
2. 编写最简 Playwright 脚本验证连接
3. 实现单个 search 任务的意图注入 + 结果提取
4. 使用 page.pause() 在 DevTools 中调试 selector

Phase 2: 状态监控增强（3-5 天）

1. 实现 SSE 流拦截逻辑
2. 添加 WebSocket 监控（Deep Research 进度）
3. 处理流式输出的累积和完成信号检测
4. 添加超时和错误处理

Phase 3: OpenClaw Skill 封装（2-3 天）

1. 定义 Skill MANIFEST（search / deep_research / agent_task）
2. 实现生命周期管理（浏览器启动检测、页面复用）
3. 添加请求队列和速率限制
4. 编写测试用例和使用文档

Phase 4: 生产优化（持续迭代）

1. 监控 Perplexity 前端更新，维护 selector 健壮性
2. 优化并发性能（多标签页复用）
3. 添加遥测和错误日志

4. 探索 Comet 其他 Agent 能力（截图理解、表单填写等）

技术依赖与环境要求

软件依赖

- **Comet Browser** — 0.1.0+ (2026年2月版本)
- **Playwright** — 1.40.0+
- **Python** — 3.10+（或 Node.js 18+）
- **OpenClaw** — 最新 main 分支

系统要求

- macOS 11+ / Windows 10+ / Linux (Ubuntu 20.04+)
- 8GB+ RAM（Deep Research 任务推荐 16GB）
- 稳定的互联网连接

Perplexity 账户

- 需要 Perplexity Pro 或 Max 订阅（访问 Deep Research 功能）
- 已在 Comet 中登录的活跃 session

参考资源

官方文档

- Perplexity Comet 官方介绍: https://www.perplexity.ai/comet/
- Chrome DevTools Protocol 文档: https://chromedevtools.github.io/devtools-protocol/
- Playwright Python API: https://playwright.dev/python/docs/api/class-playwright

社区资源

- comet-mcp-desktop (GitHub): https://github.com/AdilShaikh1/comet-mcp-desktop
- mcp-comet-browser (GitHub): https://github.com/ViktorMotor/mcp-comet-browser
- OpenClaw 官方仓库: https://github.com/openclaw/openclaw

附录 A: 完整代码示例

完整的 Skill 实现代码已开源在：

https://github.com/[your-repo]/comet-openclaw-skill

包含以下模块：

- comet_connector.py — CDP 连接管理
- intent_injector.py — 任务意图注入
- stream_monitor.py — SSE/WebSocket 监控
- result_extractor.py — 结果提取和结构化
- openclaw_skill.py — OpenClaw Skill 封装
- tests/ — 单元测试和集成测试

附录 B: 故障排查指南

常见问题

Q1: CDP 连接失败 (ERR_CONNECTION_REFUSED)

- 检查 Comet 是否以 `--remote-debugging-port=9222` 启动
- 访问 `http://localhost:9222/json` 验证端点
- 确认防火墙未阻止本地 9222 端口

Q2: 注入的任务未被执行

- 使用 `page.pause()` 打开 Playwright Inspector 检查 selector
- 确认 Comet Assistant 侧边栏已打开
- 尝试改用 `page.fill()` + `page.press('Enter')` 的标准方式

Q3: Deep Research 任务超时

- Deep Research 正常耗时 2-5 分钟，调整 timeout 至 300000ms
- 检查 Perplexity 账户是否有 Pro/Max 订阅
- 查看 Comet 中是否显示错误信息

Q4: SSE 拦截返回空数据

- 确认 route pattern 匹配 Perplexity 的实际 API endpoint
- 使用 page.on('response', lambda r: print(r.url)) 打印所有请求
- 部分响应可能使用 chunked encoding，需特殊处理

结论

本方案通过 CDP 的"意图注入 + 状态监控"模式，实现了 OpenClaw 与 Comet 浏览器的深度集成，充分利用了 Comet 的原生 Agentic Browsing 能力。相比传统的 UI 自动化方案，该架构具有更强的稳定性、更低的维护成本，以及更好的扩展性。

这种 Agent-to-Agent 的协作模式代表了未来 AI 系统集成的方向：**不是让一个 Agent 模拟另一个 Agent 的行为，而是让它们在各自的专业领域发挥所长，通过标准化接口进行协作**。

References

[1] Chrome DevTools Protocol Documentation. (2025). Chrome DevTools Protocol. https://chromedevtools.github.io/devtools-protocol/

[2] AdilShaikh1. (2026). comet-mcp-desktop: MCP Server for connecting Claude to Comet Browser. *GitHub*. https://github.com/AdilShaikh1/comet-mcp-desktop

[3] ViktorMotor. (2025). mcp-comet-browser: MCP server for controlling Comet browser via Chrome DevTools Protocol. *GitHub*. https://github.com/ViktorMotor/mcp-comet-browser

[4] Zenity Labs. (2026). Perplexity Comet: A Reversing Story. https://labs.zenity.io/p/perplexity-comet-a-reversing-story

[5] Perplexity AI. (2026). Introducing Comet: Browse at the speed of thought. https://www.perplexity.ai/hub/blog/introducing-comet

[6] Perplexity AI. (2026). Mitigating Prompt Injection in Comet. https://www.perplexity.ai/hub/blog/mitigating-prompt-injection-in-comet

[7] Perplexity AI. (2026). How We Built Security Into Comet From Day One. https://www.perplexity.ai/hub/blog/how-we-built-security-into-c

omet-from-day-one

[8] Trail of Bits. (2026). Using threat modeling and prompt injection to audit Comet. https://blog.trailofbits.com/2026/02/20/using-threat-modeling-and-prompt-injection-to-audit-comet/